

← 返回列表

skill-creator

by davepoon ☆ 2.4k 🗨 243 📄 GitHub

提供创建Claude Skills的完整框架和工具，包括模板、验证脚本和打包工具。为技能结构、资源组织和渐进式加载模式建立了清晰的规范。

GIT 历史

热门榜单

全新上线：技能 Git 版本管理 + 热门榜单，快来体验！

立即体验 →

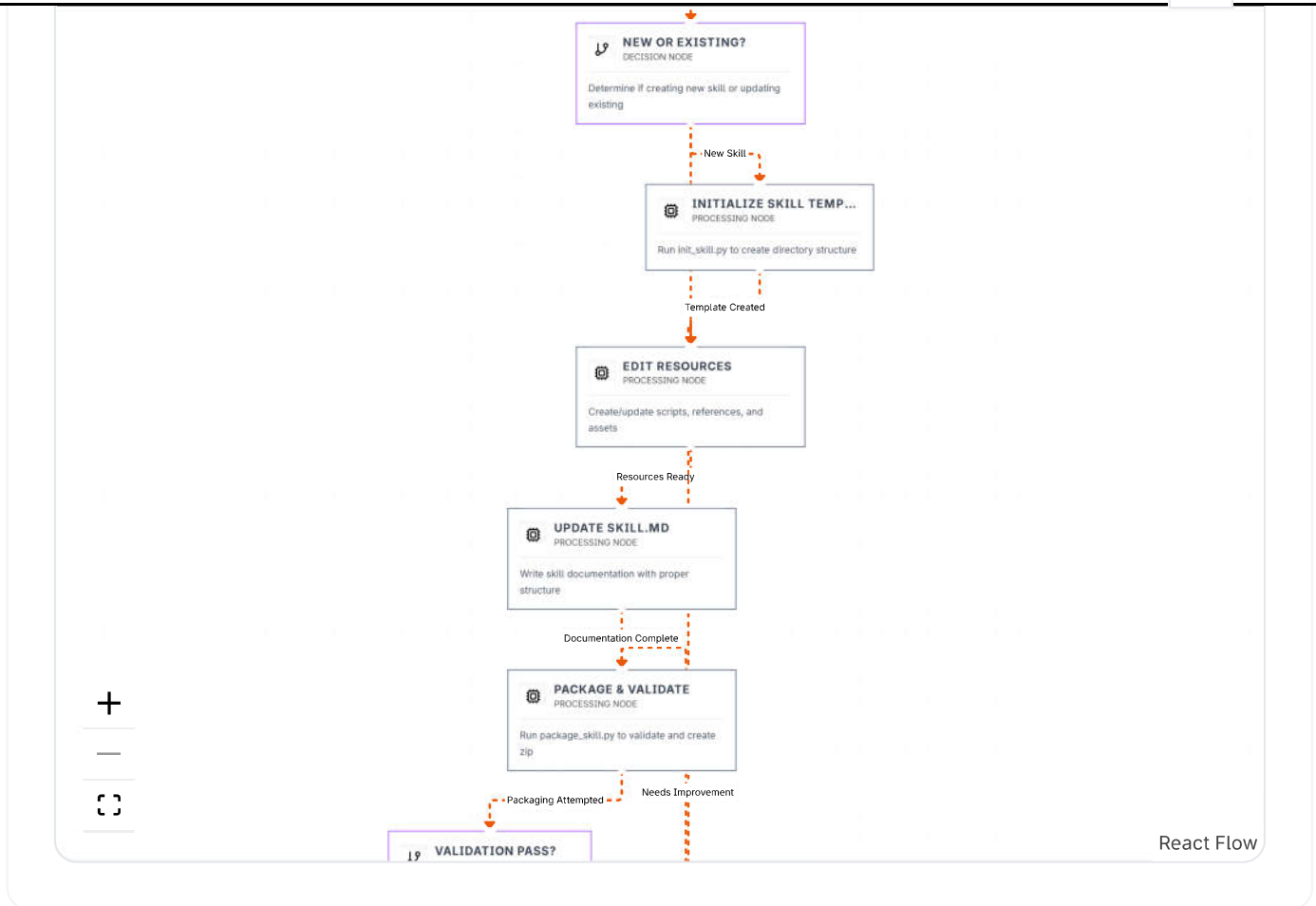
 SKILLHUB

 启动应用

登录



中文



 输出预览



📄 output_preview.md

- Always validate PDF structure before processing
- Use references/pdf_standards.md for compliance requirements
- Test scripts on sample documents before production use

Resources

- assets/fillable_form_template.pdf : Standard form template for consistent formatting
- assets/watermark_template.png : Brand watermark for document marking

Validation Report

✓ Skill validation passed ✓ YAML frontmatter complete ✓ All scripts are executable ✓ References properly organized ✓ Assets correctly placed ✓ SKILL.md follows imperative style

Package created: pdf-editor.zip (2.4 MB) Ready for distribution!

★ 适合人群

需要结构化指导和自动化工具来创建Claude Skills的开发者和技术用户



8/10 安全性

安全风险低，可放心使用

9

指令清晰度

10

实用性

9

输出质量

9

可维护性

8

创新性

元技能

skill-development

tooling

documentation

workflow

automation

COMPATIBLE AGENTS



Claude Code

~/.claude/skills/



Codex CLI

~/.codex/skills/



Gemini CLI

~/.gemini/skills/



OpenCode

~/.opencode/skill/



SKILL.md

测评详情

使用指南

SKILL.md

name: skill-creator category: development-code
description: Guide for creating effective skills. This skill should be used when users want to create a new skill (or update an existing skill) that extends Claude's capabilities with specialized knowledge, workflows, or tool integrations. license: Complete terms in LICENSE.txt

Skill Creator

This skill provides guidance for creating effective skills.

About Skills

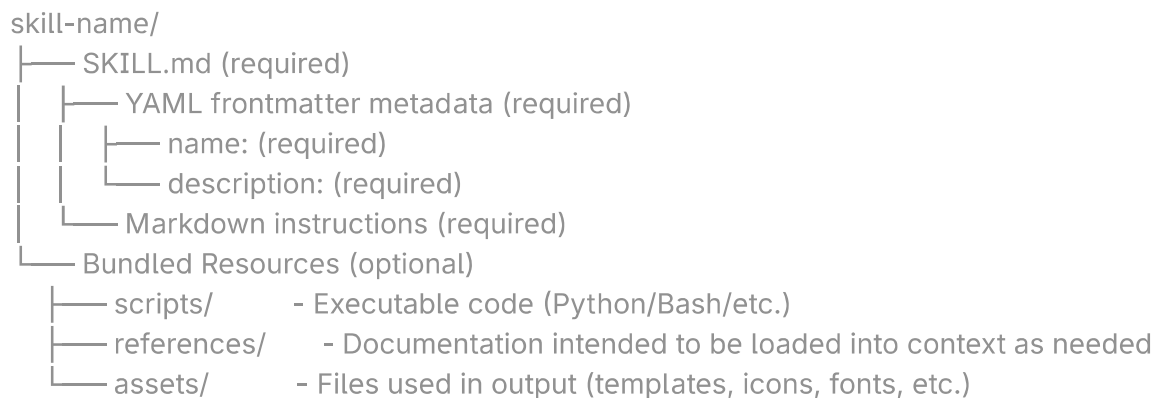
Skills are modular, self-contained packages that extend Claude's capabilities by providing specialized knowledge, workflows, and tools. Think of them as "onboarding guides" for specific domains or tasks—they transform Claude from a general-purpose agent into a specialized agent equipped with procedural knowledge that no model can fully possess.

What Skills Provide

1. Specialized workflows - Multi-step procedures for specific domains
2. Tool integrations - Instructions for working with specific file formats or APIs
3. Domain expertise - Company-specific knowledge, schemas, business logic
4. Bundled resources - Scripts, references, and assets for complex and repetitive tasks

Anatomy of a Skill

Every skill consists of a required SKILL.md file and optional bundled resources:



SKILL.md (required)

Metadata Quality: The `name` and `description` in YAML frontmatter determine when Claude will use the skill. Be specific about what the skill does and when to use it. Use the third-person (e.g. "This skill should be used when..." instead of "Use this skill when...").

Bundled Resources (optional)

Scripts (`scripts/`)

Executable code (Python/Bash/etc.) for tasks that require deterministic reliability or are repeatedly rewritten.

When to include: When the same code is being rewritten repeatedly or deterministic reliability is needed

Example: `scripts/rotate_pdf.py` for PDF rotation tasks

Benefits: Token efficient, deterministic, may be executed without loading into context

Note: Scripts may still need to be read by Claude for patching or environment-specific adjustments

References (`references/`)

Documentation and reference material intended to be loaded as needed into context to inform Claude's process and thinking.

When to include: For documentation that Claude should reference while working

Examples: `references/finance.md` for financial schemas, `references/mnda.md` for company NDA template, `references/policies.md` for company policies, `references/api_docs.md` for API specifications

Use cases: Database schemas, API documentation, domain knowledge, company policies, detailed workflow guides

Benefits: Keeps SKILL.md lean, loaded only when Claude determines it's needed

Best practice: If files are large (>10k words), include grep search patterns in SKILL.md

Avoid duplication: Information should live in either SKILL.md or references files, not both. Prefer references files for detailed information unless it's truly core to the skill—this keeps SKILL.md lean while making information discoverable without hogging the context window. Keep only essential procedural instructions and workflow guidance in SKILL.md; move detailed reference material, schemas, and examples to references files.

Assets (`assets/`)

Files not intended to be loaded into context, but rather used within the output Claude produces.

When to include: When the skill needs files that will be used in the final output

Examples: `assets/logo.png` for brand assets, `assets/slides.pptx` for PowerPoint templates, `assets/frontend-template/` for HTML/React boilerplate, `assets/font.ttf` for typography

Use cases: Templates, images, icons, boilerplate code, fonts, sample documents that get copied or modified

Benefits: Separates output resources from documentation, enables Claude to use files without loading them into context

Progressive Disclosure Design Principle

Skills use a three-level loading system to manage context efficiently:

1. **Metadata (name + description)** - Always in context (~100 words)
2. **SKILL.md body** - When skill triggers (<5k words)
3. **Bundled resources** - As needed by Claude (Unlimited*)

*Unlimited because scripts can be executed without reading into context window.

Skill Creation Process

To create a skill, follow the "Skill Creation Process" in order, skipping steps only if there is a clear reason why they are not applicable.

Step 1: Understanding the Skill with Concrete Examples

Skip this step only when the skill's usage patterns are already clearly understood. It remains valuable even when working with an existing skill.

To create an effective skill, clearly understand concrete examples of how the skill will be used. This understanding can come from either direct user examples or generated examples that are validated with user feedback.

For example, when building an image-editor skill, relevant questions include:

"What functionality should the image-editor skill support? Editing, rotating, anything else?"

"Can you give some examples of how this skill would be used?"

"I can imagine users asking for things like 'Remove the red-eye from this image' or 'Rotate this image'. Are there other ways you imagine this skill being used?"

"What would a user say that should trigger this skill?"

To avoid overwhelming users, avoid asking too many questions in a single message. Start with the most important questions and follow up as needed for better effectiveness.

Conclude this step when there is a clear sense of the functionality the skill should support.

Step 2: Planning the Reusable Skill Contents

To turn concrete examples into an effective skill, analyze each example by:

1. Considering how to execute on the example from scratch
2. Identifying what scripts, references, and assets would be helpful when executing these workflows repeatedly

Example: When building a `pdf-editor` skill to handle queries like "Help me rotate this PDF," the analysis shows:

1. Rotating a PDF requires re-writing the same code each time
2. A `scripts/rotate_pdf.py` script would be helpful to store in the skill

Example: When designing a `frontend-webapp-builder` skill for queries like "Build me a todo app" or "Build me a dashboard to track my steps," the analysis shows:

1. Writing a frontend webapp requires the same boilerplate HTML/React each time
2. An `assets/hello-world/` template containing the boilerplate HTML/React project files would be helpful to store in the skill

Example: When building a `big-query` skill to handle queries like "How many users have logged in today?" the analysis shows:

1. Querying BigQuery requires re-discovering the table schemas and relationships each time
2. A `references/schema.md` file documenting the table schemas would be helpful to store in the skill

To establish the skill's contents, analyze each concrete example to create a list of the reusable resources to include: scripts, references, and assets.

Step 3: Initializing the Skill

At this point, it is time to actually create the skill.

Skip this step only if the skill being developed already exists, and iteration or packaging is needed. In this case, continue to the next step.

When creating a new skill from scratch, always run the `init_skill.py` script. The script conveniently generates a new template skill directory that automatically includes everything a skill requires, making the skill creation process much more efficient and reliable.

Usage:

```
scripts/init_skill.py <skill-name> --path <output-directory>
```

The script:

- Creates the skill directory at the specified path

- Generates a SKILL.md template with proper frontmatter and TODO placeholders

- Creates example resource directories: `scripts/` , `references/` , and `assets/`

- Adds example files in each directory that can be customized or deleted

After initialization, customize or remove the generated SKILL.md and example files as needed.

Step 4: Edit the Skill

When editing the (newly-generated or existing) skill, remember that the skill is being created for another instance of Claude to use. Focus on including information that would be beneficial and non-obvious to Claude. Consider what procedural knowledge, domain-specific details, or reusable assets would help another Claude instance execute these tasks more effectively.

Start with Reusable Skill Contents

To begin implementation, start with the reusable resources identified above:

`scripts/` , `references/` , and `assets/` files. Note that this step may require user input. For example, when implementing a `brand-guidelines` skill, the user may need

to provide brand assets or templates to store in `assets/` , or documentation to store in `references/` .

Also, delete any example files and directories not needed for the skill. The initialization script creates example files in `scripts/` , `references/` , and `assets/` to demonstrate structure, but most skills won't need all of them.

Update SKILL.md

Writing Style: Write the entire skill using **imperative/infinite form** (verb-first instructions), not second person. Use objective, instructional language (e.g., "To accomplish X, do Y" rather than "You should do X" or "If you need to do X"). This maintains consistency and clarity for AI consumption.

To complete SKILL.md, answer the following questions:

1. What is the purpose of the skill, in a few sentences?
2. When should the skill be used?
3. In practice, how should Claude use the skill? All reusable skill contents developed above should be referenced so that Claude knows how to use them.

Step 5: Packaging a Skill

Once the skill is ready, it should be packaged into a distributable zip file that gets shared with the user. The packaging process automatically validates the skill first to ensure it meets all requirements:

```
scripts/package_skill.py <path/to/skill-folder>
```

Optional output directory specification:

```
scripts/package_skill.py <path/to/skill-folder> ./dist
```

The packaging script will:

1. **Validate** the skill automatically, checking:

YAML frontmatter format and required fields

Skill naming conventions and directory structure

Description completeness and quality

File organization and resource references

2. **Package** the skill if validation passes, creating a zip file named after the skill (e.g., `my-skill.zip`) that includes all files and maintains the proper directory structure for distribution.

If validation fails, the script will report the errors and exit without creating a package. Fix any validation errors and run the packaging command again.

Step 6: Iterate

After testing the skill, users may request improvements. Often this happens right after using the skill, with fresh context of how the skill performed.

Iteration workflow:

1. Use the skill on real tasks
2. Notice struggles or inefficiencies
3. Identify how SKILL.md or bundled resources should be updated
4. Implement changes and test again

Referenced Files

The following files are referenced in this skill and included for context.

scripts/init_skill.py

```
#!/usr/bin/env python3
"""
Skill Initializer - Creates a new skill from template

Usage:
    init_skill.py <skill-name> --path <path>
```

Examples:

```
init_skill.py my-new-skill --path skills/public
init_skill.py my-api-helper --path skills/private
init_skill.py custom-skill --path /custom/location
```

"""

```
import sys
from pathlib import Path
```

SKILL_TEMPLATE = """---

name: {skill_name}

description: [TODO: Complete and informative explanation of what the skill does and when

{skill_title}

Overview

[TODO: 1-2 sentences explaining what this skill enables]

Structuring This Skill

[TODO: Choose the structure that best fits this skill's purpose. Common patterns:

****1. Workflow-Based**** (best for sequential processes)

- Works well when there are clear step-by-step procedures
- Example: DOCX skill with "Workflow Decision Tree" → "Reading" → "Creating" → "Editing"
- Structure: ## Overview → ## Workflow Decision Tree → ## Step 1 → ## Step 2...

****2. Task-Based**** (best for tool collections)

- Works well when the skill offers different operations/capabilities
- Example: PDF skill with "Quick Start" → "Merge PDFs" → "Split PDFs" → "Extract Text"
- Structure: ## Overview → ## Quick Start → ## Task Category 1 → ## Task Category 2...

****3. Reference/Guidelines**** (best for standards or specifications)

- Works well for brand guidelines, coding standards, or requirements
- Example: Brand styling with "Brand Guidelines" → "Colors" → "Typography" → "Features"
- Structure: ## Overview → ## Guidelines → ## Specifications → ## Usage...

****4. Capabilities-Based**** (best for integrated systems)

- Works well when the skill provides multiple interrelated features
- Example: Product Management with "Core Capabilities" → numbered capability list
- Structure: ## Overview → ## Core Capabilities → ### 1. Feature → ### 2. Feature...

Patterns can be mixed and matched as needed. Most skills combine patterns (e.g., start with

Delete this entire "Structuring This Skill" section when done - it's just guidance.]

[TODO: Replace with the first main section based on chosen structure]

[TODO: Add content here. See examples in existing skills:

- Code samples for technical skills
- Decision trees for complex workflows
- Concrete examples with realistic user requests
- References to scripts/templates/references as needed]

Resources

This skill includes example resource directories that demonstrate how to organize different

scripts/

Executable code (Python/Bash/etc.) that can be run directly to perform specific operations.

Examples from other skills:

- PDF skill: `fill\_fillable\_fields.py`, `extract\_form\_field\_info.py` - utilities for PDF manipulation
- DOCX skill: `document.py`, `utilities.py` - Python modules for document processing

Appropriate for: Python scripts, shell scripts, or any executable code that performs aut

Note: Scripts may be executed without loading into context, but can still be read by Cla

references/

Documentation and reference material intended to be loaded into context to inform Claude's

Examples from other skills:

- Product management: `communication.md`, `context\_building.md` - detailed workflow guide
- BigQuery: API reference documentation and query examples
- Finance: Schema documentation, company policies

Appropriate for: In-depth documentation, API references, database schemas, compreh

assets/

Files not intended to be loaded into context, but rather used within the output Claude produ

Examples from other skills:

- Brand styling: PowerPoint template files (.pptx), logo files
- Frontend builder: HTML/React boilerplate project directories
- Typography: Font files (.ttf, .woff2)

Appropriate for: Templates, boilerplate code, document templates, images, icons, fonts

Any unneeded directories can be deleted. Not every skill requires all three types of res
"""

```
EXAMPLE_SCRIPT = '''#!/usr/bin/env python3
'''
```

Example helper script for {skill_name}

This is a placeholder script that can be executed directly.

Replace with actual implementation or delete if not needed.

Example real scripts from other skills:

- pdf/scripts/fill_fillable_fields.py - Fills PDF form fields
- pdf/scripts/convert_pdf_to_images.py - Converts PDF pages to images

```
"""

def main():
    print("This is an example script for {skill_name}")
    # TODO: Add actual script logic here
    # This could be data processing, file conversion, API calls, etc.

if __name__ == "__main__":
    main()
"""
```

EXAMPLE_REFERENCE = """# Reference Documentation for {skill_title}

This is a placeholder for detailed reference documentation.
Replace with actual reference content or delete if not needed.

Example real reference docs from other skills:

- product-management/references/communication.md - Comprehensive guide for status updates
- product-management/references/context_building.md - Deep-dive on gathering context
- bigquery/references/ - API references and query examples

When Reference Docs Are Useful

Reference docs are ideal for:

- Comprehensive API documentation
- Detailed workflow guides
- Complex multi-step processes
- Information too lengthy for main SKILL.md
- Content that's only needed for specific use cases

Structure Suggestions

API Reference Example

- Overview
- Authentication
- Endpoints with examples
- Error codes
- Rate limits

Workflow Guide Example

- Prerequisites
- Step-by-step instructions
- Common patterns
- Troubleshooting
- Best practices

"""

EXAMPLE_ASSET = """# Example Asset File

This placeholder represents where asset files would be stored.
Replace with actual asset files (templates, images, fonts, etc.) or delete if not needed.

Asset files are NOT intended to be loaded into context, but rather used within the output Claude produces.

Example asset files from other skills:

- Brand guidelines: logo.png, slides_template.pptx
- Frontend builder: hello-world/ directory with HTML/React boilerplate
- Typography: custom-font.ttf, font-family.woff2
- Data: sample_data.csv, test_dataset.json

Common Asset Types

- Templates: .pptx, .docx, boilerplate directories
- Images: .png, .jpg, .svg, .gif
- Fonts: .ttf, .otf, .woff, .woff2
- Boilerplate code: Project directories, starter files
- Icons: .ico, .svg
- Data files: .csv, .json, .xml, .yaml

Note: This is a text placeholder. Actual assets can be any file type.

"""

```
def title_case_skill_name(skill_name):
    """Convert hyphenated skill name to Title Case for display."""
    return ' '.join(word.capitalize() for word in skill_name.split('-'))
```

```
def init_skill(skill_name, path):
    """
    Initialize a new skill directory with template SKILL.md.

    Args:
        skill_name: Name of the skill
        path: Path where the skill directory should be created

    Returns:
        Path to created skill directory, or None if error
    """
    # Determine skill directory path
    skill_dir = Path(path).resolve() / skill_name

    # Check if directory already exists
    if skill_dir.exists():
        print(f"❌ Error: Skill directory already exists: {skill_dir}")
        return None

    # Create skill directory
    try:
```

```

    skill_dir.mkdir(parents=True, exist_ok=False)
    print(f"✅ Created skill directory: {skill_dir}")
except Exception as e:
    print(f"❌ Error creating directory: {e}")
    return None

# Create SKILL.md from template
skill_title = title_case_skill_name(skill_name)
skill_content = SKILL_TEMPLATE.format(
    skill_name=skill_name,
    skill_title=skill_title
)

skill_md_path = skill_dir / 'SKILL.md'
try:
    skill_md_path.write_text(skill_content)
    print("✅ Created SKILL.md")
except Exception as e:
    print(f"❌ Error creating SKILL.md: {e}")
    return None

# Create resource directories with example files
try:
    # Create scripts/ directory with example script
    scripts_dir = skill_dir / 'scripts'
    scripts_dir.mkdir(exist_ok=True)
    example_script = scripts_dir / 'example.py'
    example_script.write_text(EXAMPLE_SCRIPT.format(skill_name=skill_name))
    example_script.chmod(0o755)
    print("✅ Created scripts/example.py")

    # Create references/ directory with example reference doc
    references_dir = skill_dir / 'references'
    references_dir.mkdir(exist_ok=True)
    example_reference = references_dir / 'api_reference.md'
    example_reference.write_text(EXAMPLE_REFERENCE.format(skill_title=skill_title))
    print("✅ Created references/api_reference.md")

    # Create assets/ directory with example asset placeholder
    assets_dir = skill_dir / 'assets'
    assets_dir.mkdir(exist_ok=True)
    example_asset = assets_dir / 'example_asset.txt'
    example_asset.write_text(EXAMPLE_ASSET)
    print("✅ Created assets/example_asset.txt")
except Exception as e:
    print(f"❌ Error creating resource directories: {e}")
    return None

# Print next steps
print(f"\n✅ Skill '{skill_name}' initialized successfully at {skill_dir}")
print("\nNext steps:")
print("1. Edit SKILL.md to complete the TODO items and update the description")

```

```
print("2. Customize or delete the example files in scripts/, references/, and assets/")
print("3. Run the validator when ready to check the skill structure")

return skill_dir

def main():
    if len(sys.argv) < 4 or sys.argv[2] != '--path':
        print("Usage: init_skill.py <skill-name> --path <path>")
        print("\nSkill name requirements:")
        print(" - Hyphen-case identifier (e.g., 'data-analyzer')")
        print(" - Lowercase letters, digits, and hyphens only")
        print(" - Max 40 characters")
        print(" - Must match directory name exactly")
        print("\nExamples:")
        print("  init_skill.py my-new-skill --path skills/public")
        print("  init_skill.py my-api-helper --path skills/private")
        print("  init_skill.py custom-skill --path /custom/location")
        sys.exit(1)

    skill_name = sys.argv[1]
    path = sys.argv[3]

    print(f"🚀 Initializing skill: {skill_name}")
    print(f"  Location: {path}")
    print()

    result = init_skill(skill_name, path)

    if result:
        sys.exit(0)
    else:
        sys.exit(1)

if __name__ == "__main__":
    main()
```

scripts/package_skill.py

```
#!/usr/bin/env python3
"""
Skill Packager - Creates a distributable zip file of a skill folder

Usage:
python utils/package_skill.py <path/to/skill-folder> [output-directory]
```


Example:

```
python utils/package_skill.py skills/public/my-skill
python utils/package_skill.py skills/public/my-skill ./dist
```

```
"""
```

```
import sys
```

```
import zipfile
```

```
from pathlib import Path
```

```
from quick_validate import validate_skill
```

```
def package_skill(skill_path, output_dir=None):
```

```
    """
```

```
    Package a skill folder into a zip file.
```

```
    Args:
```

```
        skill_path: Path to the skill folder
```

```
        output_dir: Optional output directory for the zip file (defaults to current directory)
```

```
    Returns:
```

```
        Path to the created zip file, or None if error
```

```
    """
```

```
    skill_path = Path(skill_path).resolve()
```

```
    # Validate skill folder exists
```

```
    if not skill_path.exists():
```

```
        print(f"❌ Error: Skill folder not found: {skill_path}")
```

```
        return None
```

```
    if not skill_path.is_dir():
```

```
        print(f"❌ Error: Path is not a directory: {skill_path}")
```

```
        return None
```

```
    # Validate SKILL.md exists
```

```
    skill_md = skill_path / "SKILL.md"
```

```
    if not skill_md.exists():
```

```
        print(f"❌ Error: SKILL.md not found in {skill_path}")
```

```
        return None
```

```
    # Run validation before packaging
```

```
    print(f"🔍 Validating skill...")
```

```
    valid, message = validate_skill(skill_path)
```

```
    if not valid:
```

```
        print(f"❌ Validation failed: {message}")
```

```
        print(" Please fix the validation errors before packaging.")
```

```
        return None
```

```
    print(f"✅ {message}\n")
```

```
    # Determine output location
```

```
    skill_name = skill_path.name
```

```
    if output_dir:
```

```
        output_path = Path(output_dir).resolve()
```

```
    output_path.mkdir(parents=True, exist_ok=True)
else:
    output_path = Path.cwd()

zip_filename = output_path / f"{skill_name}.zip"

# Create the zip file
try:
    with zipfile.ZipFile(zip_filename, 'w', zipfile.ZIP_DEFLATED) as zipf:
        # Walk through the skill directory
        for file_path in skill_path.rglob('*'):
            if file_path.is_file():
                # Calculate the relative path within the zip
                arcname = file_path.relative_to(skill_path.parent)
                zipf.write(file_path, arcname)
                print(f" Added: {arcname}")

    print(f"\n✅ Successfully packaged skill to: {zip_filename}")
    return zip_filename

except Exception as e:
    print(f"❌ Error creating zip file: {e}")
    return None

def main():
    if len(sys.argv) < 2:
        print("Usage: python utils/package_skill.py <path/to/skill-folder> [output-directory]")
        print("\nExample:")
        print(" python utils/package_skill.py skills/public/my-skill")
        print(" python utils/package_skill.py skills/public/my-skill ./dist")
        sys.exit(1)

    skill_path = sys.argv[1]
    output_dir = sys.argv[2] if len(sys.argv) > 2 else None

    print(f"📦 Packaging skill: {skill_path}")
    if output_dir:
        print(f" Output directory: {output_dir}")
    print()

    result = package_skill(skill_path, output_dir)

    if result:
        sys.exit(0)
    else:
        sys.exit(1)

if __name__ == "__main__":
```

```
main()
```

内容来源: <https://github.com/davepoon/buildwithclaude#plugins~all-skills~skills~skill-creator>
本站仅做整理展示，版权归原作者所有

S级

9.1
AI SCORE

✓ Best Practices

最佳实践: F

📁 下载完整 Skill

📄 仅复制 SKILL.md

> 一键安装

▶ 试用此 Skill

🤍 0

用户评价

登录后可评价此 Skill

USER RATING

 UP

 DOWN

文件浏览5

scripts

init_skill.py

package_skill.py

quick_validate.py

LICENSE.txt

SKILL.md



发现、浏览、一键复制最优质的 Agent Skills。15,000+ 精选技能，LLM 测评打分，助你效率翻倍。

订阅更新

输入你的邮箱

快速链接

全部 Skills

排行榜

API DocsNEW

关于

GitHub

Claude 文档

服务条款

联系

 反馈 Bug

 联系我们

内容来源于 GITHUB，版权归作者所有

© 2026 SKILLHUB. ALL RIGHTS RESERVED.